Design and FPGA Realization of QUBO hardware accelerator for MAX-CUT problem

Maciej Lewandowski
Department of Electrical and Electronic Engineering
Univeristy of Manchester
Manchester, UK
maciej.lewandowski@student.manchester.ac.uk

Piotr Dudek
Department of Electrical and Electronic Engineering
University of Manchester
Manchester, UK
p.dudek@manchester.ac.uk

***Abstract*—Combinatorial optimization problems are fundamental to many industrial and scientific applications, but are often NP-hard, requiring heuristic and probabilistic approaches to find good solutions fast. This paper presents the first hardware implementation of the neuromorphic-inspired NeuroSA algorithm, for accelerating Quadratic Unconstrained Binary Optimization (QUBO) problems. We showed it is possible to run standard a benchmark QUBO problem with 800 variables on a low-cost FPGA with clock speed of approximately 144 MHz.**

***Index Terms*—Ising Model, QUBO, FPGA, Neuromorphic Engineering**

I. Introduction

Solving combinatorial optimization problems is a fundamental challenge across a vast number of applications, including all CAD stages of VLSI design [1], optimal PCB assembly planning [2], efficient scheduling of aircraft arrivals in busy airports [3], finding optimal routes of delivery vehicles [4] or DNA sequencing [5]. These optimization problems are NP-hard problems which cannot be solved exactly using deterministic algorithms in reasonable time even for relatively small problem sizes. These problems are commonly encountered in real world applications with most well-known instances such as Travelling Salesman Problem, Boolean Satisfiability problem or MAX-CUT problem [7]. Due to common occurrence of these problems in many industrial and scientific fields [1]–[6], a significant effort has been put into developing many heuristic algorithms [1], [5], [7], often based on Monte-Carlo methods [6], [7], [10] and dedicated hardware accelerators [8], to find good solutions to these problems quickly. Two particularly interesting NP-hard problems are the Quadratic Unconstrained Binary Optimization (QUBO) problem and Ising model, which are closely related by a simple transformation [16]. These two problems have received a lot of attention [8]–[10] due to their well-developed theory and connection to physics [6], [15], [16]. This is due to a well-known fact from complexity theory that it is possible to transform in polynomial time any problem from NP complexity class to these two problems [16], which means that having algorithms or hardware accelerating these two problems, would mean accelerating many difficult problems with real-world applications.

One approach to accelerate QUBO problems is the NeuroSA [14] algorithm. The approach was inspired by inherently-parallel, neuromorphic architectures which can leverage noise and non-linear dynamics. Authors also demonstrated implementation of the algorithm on the SpiNNaker2 system, demonstrating feasibility of this approach at large scales. This work builds on the NeuroSA algorithm presented in [14] and to our knowledge introduces first hardware architecture of the NeuroSA with distributed memory storage and implements it on an FPGA. We demonstrate the performance of the architecture on MAX-CUT problems, report performance and the FPGA blocks required to implement it, and provide a discussion on further improvements.

The rest of the paper is structured as follows: the background information on the implemented algorithm is discussed in Section II, details about the implemented architecture can be found in Section III, while the results and comparison to software simulation can be found in Section IV.

II. Background

The QUBO problem is often formulated as follows [9]: let $\mathbf{Q}$ be a symmetric, integer or real $N \times N$ matrix, $\mathbf{s}$ is a N-dimensional binary vector, where each element $s[k]$ is 0 or 1 and $\mathbf{b}$ is a N-dimensional vector, which represents a bias vector. The problem of QUBO is to find the vector $\mathbf{s}^*$, that maximizes the objective function $H(\mathbf{s})$:

$$\begin{aligned} H(\mathbf{s}) &= \mathbf{s}^T\mathbf{Q}\mathbf{s} + \mathbf{b}^T\mathbf{s} \\ \mathbf{s}^* &= \arg\max_{\mathbf{s}} H(\mathbf{s}) \end{aligned} \tag{1}$$

The Ising model has very similar formulation, with the main distinction that every value of $s[k] \in \{-1, 1\}$, representing spin orientations, and the problem is formulated as finding minimum of the "energy" function H(**s**).

In real-world applications, N can often reach thousands, making it impossible to systematically explore even a small fraction of the entire solution space. To overcome this challenge, probabilistic and approximate algorithms are often used which produce sufficiently good solutions, however do not guarantee finding the global minimum. A widely used approach is Simulated Annealing (SA) algorithm [12]. The SA algorithm was inspired by annealing process in physics and the key idea behind it, is to stochastically perturb the

solution until a good solution is obtained. The algorithm has a parameter called "temperature" which controls the probability of accepting some proposed perturbation to the system. It starts in a high temperature region, where there is a large probability for the system to change, even if that change doesn't improve the solution. As the simulation progresses, the temperature decreases, and the probability to accept changes becomes much smaller. More specifically, the temperature parameter T_t is a sequence of strictly positive, non-increasing numbers, with a limit $\lim_{t\to\infty} T_t = 0$. Then the algorithm produces a sequence of states $\mathbf{s}_0$, $\mathbf{s}_1$, $\mathbf{s}_2$, ..., where the algorithm proposes a new state $\mathbf{s}'_t$, which is accepted with a certain probability. This is captured by the following equation:

$$\mathbf{s}'_{t+1} = \begin{cases} \mathbf{s}'_t \text{ with probability } exp(\frac{-[H(\mathbf{s}'_t)-H(\mathbf{s}_t)]}{T_t}), \\ \mathbf{s}_t \text{ otherwise} \end{cases} \tag{2}$$

It also has been proven, that following a particular annealing schedule will result in asymptotic convergence of the algorithm to the ground state [13].
These equations form the starting point for the NeuroSA architecture [14]. The algorithm makes the following assumptions:

- The authors replaced the external bias vector **b** with **0** and modeled this as additional constraints to the **Q** matrix.
- The vector **s** is described as consisting of pairs of asynchronous ON-OFF neurons, meaning that spikes of one neuron $s_+[k]$, correspond to ON events, and spikes of the other neuron, $s_-[k]$ correspond to OFF events. They are cross-coupled and mutually exclusive, only one of them is active at given instance of time. The neurons are connected to every other neuron pair through the synaptic weights Q_{kj},–Q_{kj}. The authors also introduce the membrane voltage $V_{mem}[k]$, which represents an internal state of a neuron and affects the probability of the neuron pair to "flip".
- An ideal asynchronous operation is assumed, meaning only a single neuron changes at one instance of time. The threshold for the these neurons to fire is dynamically adjusted by a "Fowler-Nordheim Annealer" which uses a temperature-controlled exponentially-distributed noise source.

The details of the NeuroSA algorithm were provided in [14]. We deviate from the original notation and make a following simplification with the state of pair of neurons : because only one of them can be ON and other is OFF, we simply define $s[k] \in \{-1, 1\}$ as the state of the neuron pair. In the presented digital implementation of the algorithm, we follow derivations provided in [14] to reformulate (2) to arrive at a following state change rule

$$s_{t+1}[k^*] = \begin{cases} -s_t[k^*] \text{ if } \frac{d \cdot log(\beta U[0,1]+\epsilon)}{log(1+\alpha T)} < s_t[k^*] \cdot V_{mem}[k^*] \\ s_t[k^*] \text{ otherwise} \end{cases} \tag{3}$$

Where k^* is some randomly chosen neuron, U[0,1] denotes a random real number generated in range [0, 1], and α, β, d are user-controlled hyperparameters.

III. Proposed Hardware Architecture

The architecture of the proposed system is illustrated in Fig. 1. In the design we assume **Q** to be a binary matrix, representing edges between vertices i and j. The size of the problem and size of the matrix is denoted as N. We denote A as the number of neurons in each Unit, and for efficient indexing, $A = 2^{A'}$ with $A' \in \mathbb{N}$. In the whole architecture there are M such units. We partition the problem of size N into M units, each containing A units, therefore $N = M \cdot 2^{A'}$. Similarly to approach taken in NeuroSA [14], the proposed architecture has been evaluated on MAX-CUT problems, due to their well-established benchmarks. The architecture should be able to scale to integer or potentially limited floating point weights which would broaden significantly the scope of solvable problems. This however would significantly increase the complexity of membrane updates and would require much higher resource utilization than allowed by the FPGA used for prototyping.

A. Architecture Overview

We divide the solution space $(\mathbf{s}, \mathbf{V_{mem}})$ into independent units, denoted in blue in Fig. 1(a). Every unit contains a memory block for A membranes and states, a "Q-Matrix Fragment" containing partial information about the problem (A columns of **Q** matrix), "Membrane update" circuitry, neuron selection circuits, and computing circuits (a "Fowler-Nordheim Noise Generator" and a comparator). The flowchart for the architecture can be found in Fig. 1(b). Firstly, the unit u will fetch the membrane $V_{mem}[k_u]$ and the state $s[k_u]$ of a randomly selected neuron k_u and generate a random value following a required distribution multiplied by the temperature term [14]. In next stage we compare these values: we transmit whether the comparison succeeded, information about the membrane, state, index and unit number to the Spike Selector. Comparison generating 1 means that the neuron that was selected should change its state (from ON to OFF, or OFF to ON). The Spike Selector will select a neuron from one of the units that requested such a "flip", assert signal $fired$, and pass the neurons global index k^*, its membrane $V_{mem}[k*]$ and state $s[k*]$, to the Scheduler. When the $fired$ signal is asserted, the Scheduler will pause the generation process of units for 3 clock cycles which is required to allow the neuron updates to complete. It will transmit the global index of neuron that fired with a signal $update$ to the units. The Scheduler also implements the temperature scheduling: after a set number of clock cycles, it will send the $decrease\ temp$ signal which will lower the "temperature" of the unit, leading to less frequent changes. After $update$ and index k^* are transmitted, the units will stop generating samples, to allow time for membranes to get updated. The units will fetch appropriate row from the "Q-Matrix Fragment" and in the next clock cycle, perform parallel updates to membranes and state (if appropriate). After the membrane updates, the units go back to generating samples and the process repeats.

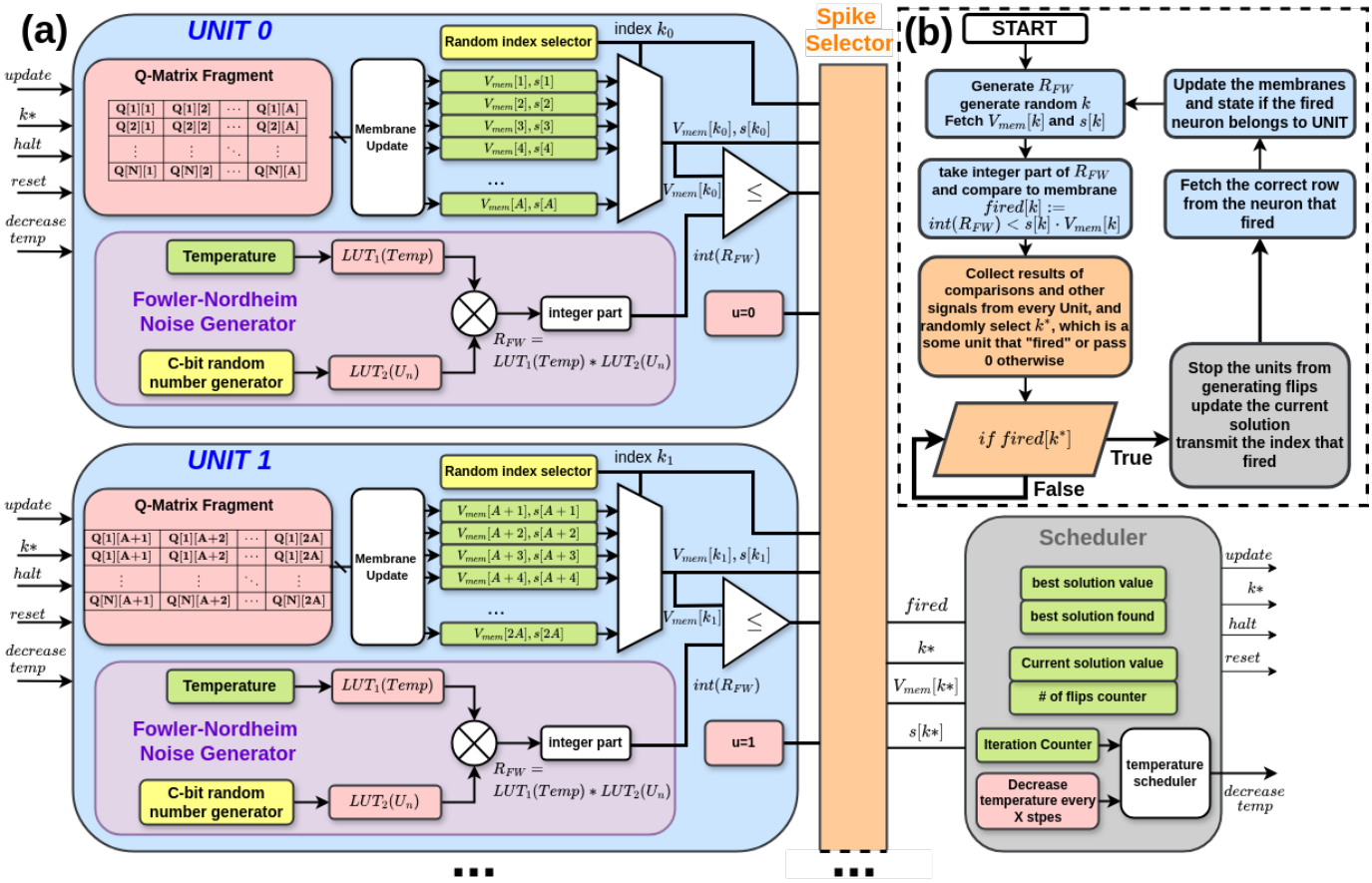


Fig. 1. (a) the diagram of the proposed architecture. Units are marked in blue color, Computing circuit is marked in purple, green corresponds to read and write variables, red corresponds to memories programmed once and stay the same during running the computation, yellow corresponds to random number generators, spike selector is marked in orange and scheduler is marked in gray. (b) contains the flowchart of the computation

B. Design choices

In this section we describe the design choices made and the motivation behind them.

The key aspect of the design is memory proximity to computation. In this architecture there is no global storage (except for a control processor which partitions **Q** and uploads it to the proposed accelerator). Each unit depends only on a small portion of the matrix **Q**, and stores the membrane data for its neurons. This allows for the use of small, cost-effective, decentralized memory blocks within the units. For the largest problem we tried to solve, with $N = 800$ and $A = 32$, and with the number of bits required for the lookup tables being 2048 bits, it is possible to fit the required 29696 bits into a single Xilinx BRAM block [17]. Since all elements of **Q** are binary variables, membrane updates are simple: the row corresponding to index k^* selected by the Spike Selector is fetched, and updates to the membranes are performed in the next clock cycle. These updates are performed in parallel, following the equation $\Delta V_{mem}[i] = -2s'[k^*]Q[k^*, i]$, where $s'[k^*]$ is previous state (–1 or 1), before change.

Initially all real numbers were represented using 16-bit floating point, because it allowed easier comparison with the Python-based numerical simulation. Fixed point representation was used in the final implementation, for more efficient hardware implementation. The computation of (3) is divided into two parts. First part is a random term which corresponds to $d \cdot log(\beta \cdot U[0,1] + \epsilon)$, which has been implemented as a C-bit Linear-Feedback Shift Registers (LFSR) generating random integer $u_n = U[0, 2^C - 1]$. This value is passed to the lookup table LUT_2 which takes the C-bit integer input, and outputs 16-bit real value of the given equation. Similarly, the system temperature T is implemented as a 7-bit integer representing current temperature of the system and is passed to LUT_1, which stores pre-computed values for $\frac{1}{log(1+\alpha T)}$. The temperature signal is reduced over time using an external "decrease temp" signal. The temperature signal per unit can be seen as redundant in the implemented algorithm, as the temperature is a global property of the system. However, in future work, implementing unit-specific temperature could enable implementing the parallel-tempering algorithm, which can result in better performance [11].

As discussed in Section II, random number generation is critical for the simulated annealing. We experimented with LFSR, which are often used in hardware design due to their

cheap implementation, Halton sequences used in accelerating Monte Carlo simulations [19], and standard Mersenne-Twister [20]. All of them were implemented into Python-based solver, and yielded solutions close to state-of-the-art (SOTA) on tested problems. We run the simulation 20 times on problem G15 [21] and the results of running the simulation can be found in Table I. Based on simulation results, we chose 7-bit LFSR for computation as it provided a good balance between performance, small resource utilization for LFSR and the size of the lookup table is fairly small (2048 bits).

As the number of units increases, the Spike Selector needs to be carefully designed. In the presented implementation, a simple mechanism is used: the highest priority is given to unit 0, then second-highest priority is given to unit 1, and the smallest priority is given for unit M-1. Despite the simplicity of the selector, Python simulation still was able to achieved solutions close to the SOTA. Further experimentation was done with a round-robin schedule, where after the 1st "update" signal, Unit 1 will have highest priority, after the 2nd "update" signal, Unit 2 will have highest priority and so on. The Scheduler is a small block, whose job is to transmit which global index k^* flipped, decrease temperature every number of clock cycles and save and transmit the best solution.

TABLE I
PERFORMANCE EVALUATION OF RANDOM NUMBER GENERATORS, PROBLEM=G15 [21], N=800, 25 UNITS, SIMPLE SELECTION, 7-BIT TEMPERATURE, BEST SOLUTION KNOWN = 3050

Metric	**Random Number Generators (20 simulations)**				
	6-bit LFSR	***7-bit LFSR***	***8-bit LFSR***	***Mersenne Twister***	***Halton Seq.***
Average Distance from SOTA	5.05	2.4	1.4	1.0	1.0
Best Solution	3049	3049	3049	3049	3049
Best Solution Achieved	1/20 (5.0%)	18/20 (90%)	19/20 (95%)	20/20 (100%)	20/20 (100%)

IV. IMPLEMENTATION AND RESULTS

The software used for the implementation was Vivado 2024.1, the design was implemented using SystemVerilog and was synthesized to a low cost Xilinx XC7Z020 board. The problem we tried to solve had 800 variables (N=800), and we partitioned it into units containing 8 neurons (A=8), 16 neurons (A=16) and 32 neurons (A=32). The number of elements for a single synthesized unit and the whole design is presented in Table II. The top row in each cell represents resources required to construct a single unit with percentage of total usage in parentheses, while the second number represents resources required to construct the entire design. From the synthesis we also obtained the critical path (the largest data path delay), denoted as t_c.

Given the results of the synthesis, it is possible to run a fairly large problem of N = 800 on the low cost FPGA board. The problem G15 [21] with the size of the problem of 800,

TABLE II
FPGA BLOCKS REQUIRED FOR DIFFERENT UNITS FROM SYNTHESIS USING VIVADO 2024.1

Parameters	Utilization For	**FPGA elements required**				
		LUT	***LUTRAM***	***FLIP FLOPS***	***BRAM***	***DSP***
N=800, A=8; t_c= 16.013ns	Single Unit	241 (0.45%)	8 (0.05%)	208 (0.2%)	1.5 (1.07%)	1 (0.45%)
	Whole Design	24594 (46.23%)	800 (4.6%)	19702 (18.52%)	99 (70.71%)	100 (45.5%)
N=800, A=16; t_c= 9.361ns	Single Unit	368 (0.69%)	8 (0.05%)	293 (0.28%)	1.5 (1.07%)	1 (0.45%)
	Whole Design	18738 (35.22%)	400 (2.3%)	14112 (13.26%)	51 (36.43%)	50 (22.73%)
N=800, A=32; t_c= 6.937ns	Single Unit	656 (1.23%)	8 (0.05%)	458 (0.43%)	2 (1.43%)	1 (0.45%)
	Whole Design	16346 (30.73%)	200 (1.15%)	11230 (10.55%)	39 (27.86%)	25 (11.36%)

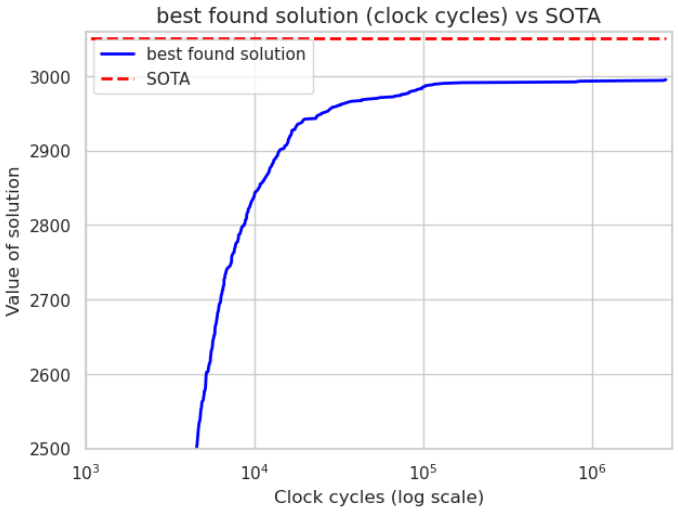


Fig. 2. Recorded value of best solution from postsynthesis simulation, of problem for N=800, A=32, 25 Units, 7-bit temperature, 7-bit Random Number generator

with 25 units (A=32) was run, and results can be seen in Fig. 2.

Within 10^5 clock cycles (0.7ms, with $t_{critical} = 6.937ns$), the implementation achieved a solution of 2986, and after $2.75 \cdot 10^6$ cycles (19.08 ms) it settles on 2995 which is 54 away from SOTA. This is a promising starting point of energy-efficient QUBO solvers and warrants further investigation on the implementation of the NeuroSA algorithm on FPGAs.

ACKNOWLEDGMENT

The authors extend gratitude to the 2024 Telluride Neuromorphic Cognition Engineering Workshop, where the NeuroSA was developed, with special thanks to Johannes Leugering for the presentation and providing the Python code that formed the foundation of the implementation. M.L. extends gratitude to Minh Vu, Rawan Elsayed, and Revathy Ravuri for their contributions to discussions and assistance with proofreading.

REFERENCES

[1] B. Korte, D. Rautenbach, and J. Vygen, 'BonnTools: Mathematical Innovation for Layout and Timing Closure of Systems on a Chip', Proceedings of the IEEE, vol. 95, no. 3, pp. 555–572, Mar. 2007, doi: 10.1109/JPROC.2006.889373.

[2] L. P. Khoo and T. K. Ng, "A genetic algorithm-based planning system for PCB component placement," International Journal of Production Economics, vol. 54, no. 3, pp. 321–332, May 1998, doi: 10.1016/S0925-5273(98)00010-3.

[3] A. P. Saraf and G. L. Slater, "An efficient combinatorial optimization algorithm for optimal scheduling of aircraft arrivals at congested airports," in 2006 IEEE Aerospace Conference, Mar. 2006, p. 11 pp.-. doi: 10.1109/AERO.2006.1655877.

[4] B. Golden, S. Raghavan, and E. Wasil, Eds., The Vehicle Routing Problem: Latest Advances and New Challenges, vol. 43. in Operations Research/Computer Science Interfaces, vol. 43. Boston, MA: Springer US, 2008. doi: 10.1007/978-0-387-77778-8.

[5] J. Błażewicz, P. Formanowicz, and M. Kasprzak, "Selected combinatorial problems of computational biology," European Journal of Operational Research, vol. 161, no. 3, pp. 585–597, Mar. 2005, doi: 10.1016/j.ejor.2003.10.054.

[6] S. Tanaka, Y. Matsuda, and N. Togawa, "Theory of Ising Machines and a Common Software Platform for Ising Machines," in 2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC), Jan. 2020, pp. 659–666. doi: 10.1109/ASP-DAC47756.2020.9045126.

[7] B. Korte and J. Vygen, Combinatorial Optimization: Theory and Algorithms, vol. 21. in Algorithms and Combinatorics, vol. 21. Berlin, Heidelberg: Springer, 2018. doi: 10.1007/978-3-662-56039-6.

[8] S. Matsubara et al., "Ising-Model Optimizer with Parallel-Trial Bit-Sieve Engine," in Complex, Intelligent, and Software Intensive Systems, L. Barolli and O. Terzo, Eds., Cham: Springer International Publishing, 2018, pp. 432–438. doi: 10.1007/978-3-319-61566-0_39.

[9] A. P. Punnen, Ed., The Quadratic Unconstrained Binary Optimization Problem: Theory, Algorithms, and Applications. Cham: Springer International Publishing, 2022. doi: 10.1007/978-3-031-04520-2.

[10] Y.-T. Kao, J.-L. Liao, and H.-C. Hsu, "Solving Combinatorial Optimization Problems on Fujitsu Digital Annealer," Proceedings of 2023 TANET & NCS, arXiv.org. Accessed: Oct. 09, 2024. [Online]. Available: https://arxiv.org/abs/2311.05196v1

[11] M. Aramon, G. Rosenberg, E. Valiante, T. Miyazawa, H. Tamura, and H. G. Katzgraber, "Physics-Inspired Optimization for Quadratic Unconstrained Problems Using a Digital Annealer," Front. Phys., vol. 7, Apr. 2019, doi: 10.3389/fphy.2019.00048.

[12] "Optimization by Simulated Annealing — Science." Accessed: Oct. 17, 2024. [Online]. Available: https://www.science.org/doi/10.1126/science.220.4598.671

[13] B. Hajek, "COOLING SCHEDULES FOR OPTIMAL ANNEALING"., MATHEMATICS OF OPERATIONS RESEARCH, Vol. 13. No. 2. May 1911

[14] Z. Chen et al., "ON-OFF Neuromorphic ISING Machines using Fowler-Nordheim Annealers," arXiv.org. unpublished. Accessed: Oct. 09, 2024. [Online]. Available: https://arxiv.org/abs/2406.05224v1

[15] N. Mohseni, P. L. McMahon, and T. Byrnes, "Ising machines as hardware solvers of combinatorial optimization problems," Nat Rev Phys, vol. 4, no. 6, pp. 363–379, Jun. 2022, doi: 10.1038/s42254-022-00440-8.

[16] A. Lucas, "Ising formulations of many NP problems," arXiv.org. Accessed: Oct. 09, 2024. [Online]. Available: https://arxiv.org/abs/1302.5843v3

[17] "Zynq-7000 SoC Data Sheet: Overview (DS190)," 2018.

[18] Y. Shao et al., "Probabilistic computing with voltage-controlled dynamics in magnetic tunnel junctions," Nanotechnology, vol. 34, no. 49, p. 495203, Sep. 2023, doi: 10.1088/1361-6528/acf6c7.

[19] C. Joy, P. P. Boyle, and K. S. Tan, "Quasi-Monte Carlo Methods in Numerical Finance," Management Science, vol. 42, no. 6, pp. 926–938, Jun. 1996, doi: 10.1287/mnsc.42.6.926.

[20] M. Matsumoto and T. Nishimura, "Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator," ACM Trans. Model. Comput. Simul., vol. 8, no. 1, pp. 3–30, Jan. 1998, doi: 10.1145/272991.272995.

[21] MAX-CUT problem set, Accessed: Oct. 18, 2024. [Online]. Available: https://web.stanford.edu/ yyye/yyye/Gset/